The Blind Architects

Malware Traffic Analysis Tool

Written Project Report

Caleb Meister, Nathan Sigulas, Campbell Russo

Introduction

One of the biggest challenges in digital forensics is dealing with the sheer volume of network data that analysts have to sift through during an investigation. When a network is compromised, the evidence isn't in one place. It shows up in DNS queries, HTTP headers, TCP connection logs, and raw packet payloads all at the same time. Trying to manually correlate all of that across hundreds of thousands of log lines is slow, and it's easy to miss things. We wanted to build something that could automate that process and make it faster to identify what actually matters.

Our tool, the Blind Architects Malware Traffic Analysis Tool, takes network log files or PCAP files as input and scans them for indicators of compromise across six detection areas: IP reputation, domain reputation, port usage, DNS anomalies, HTTP patterns, and payload signatures. Every finding gets tagged with a MITRE ATT&CK technique ID so there's immediate context for whatever is flagged. After the scan finishes, the tool sends all the findings to an AI model that returns a plain-English summary of what the traffic suggests and what actions to take. The goal was to make something that's useful whether you're an experienced analyst or still learning.

Our team split responsibilities clearly. Caleb Meister was the lead developer and wrote all of the source code, including the detection engine, the binary PCAP parser, the threat intelligence layer, and the reporting system. Campbell Russo handled documentation, the written report, and researched the digital forensics concepts and MITRE ATT&CK techniques that shaped the tool's design. Nathan Sigulas handled the project presentation and contributed research on existing malware analysis tools, attack pattern documentation, and how real-world SOC teams use frameworks like MITRE ATT&CK, which helped guide a lot of our design decisions. The tool is written entirely in Python 3.8 or higher, doesn't require any third-party libraries for its core functionality, and is publicly available at github.com/MeistCaleb/The_Blind_Artitech_IT360_Proj.

Technical Implementation

We made a deliberate choice early on to build the entire tool using only Python's standard library. No pip installs, no external dependencies. It made development harder in some ways, but it also means the tool can run anywhere Python 3.8 is installed without any setup. The modules we relied on most were re for regex matching, math for entropy calculations, struct and socket for PCAP parsing, ipaddress for IP validation, argparse for the CLI, json and urllib.request for the AI API calls, pathlib for file handling, and dataclasses for the data model.

The foundation of the codebase is an abstract base class in core/detectors/base.py. The BaseDetector class defines a single abstract method, analyze(line: str) -> List[Finding], that every detector has to implement. It also has a _finding() helper method that constructs Finding objects and automatically truncates the value field to 120 characters so nothing produces excessively long output. All six detectors inherit from this class, which kept the interface consistent and made it easy to add or modify detectors without touching the rest of the pipeline.

The data model lives in core/models.py and uses Python dataclasses. A Finding object stores the severity level (critical, high, medium, or low), the ioc_type string like MALICIOUS_IP or POWERSHELL_ENCODED_CMD, the raw indicator value, a human-readable detail string, and an optional MITRE technique ID. Findings implement __lt__ using a severity order dictionary so they can be sorted with critical findings first. A LogLine pairs a line number and raw text with its list of findings. AnalysisResults wraps everything together with a statistics dictionary and exposes a filter_by_severity method for filtering output down to specific levels.

The main analysis engine is in core/analyzer.py. The TrafficAnalyzer class instantiates all six detectors at startup and has two public methods. analyze_file checks whether the input is a binary PCAP file or plain text using the is_pcap helper, converts PCAP files to log line strings using PCAPReader, and then passes everything to analyze_text. That method splits the input into lines, skips blanks and comment lines starting with #, and runs each line through every detector. Each detector call is wrapped in a try-except so a bug in one detector can't crash the whole run. Findings from all detectors get collected, deduplicated on the ioc_type and value combination so the same indicator isn't reported twice, sorted by severity, and stored in a LogLine. Statistics are accumulated as the analysis runs.

The PCAP parser in core/pcap_reader.py was probably the most technically involved part of the project. We implemented it from scratch using struct and socket with no libpcap or Scapy. It detects the file format by reading the first four bytes and comparing against five magic numbers: 0xa1b2c3d4 for little-endian classic pcap, 0xd4c3b2a1 for big-endian, 0xa1b23c4d and 0x4d3cb2a1 for nanosecond timestamp variants, and 0x0a0d0d0a for pcapng. For classic pcap it reads the 24-byte global header to get the byte order and link-layer type, then walks through packet records by reading the 16-byte record header and slicing out the captured data. For pcapng it iterates through variable-length block structures, pulling link layer type from Interface Description Blocks and packet data from Enhanced Packet Blocks and Simple Packet Blocks.

Packet dissection handles Ethernet frames (link type 1), raw IP (link type 101), and Linux cooked captures (link type 113). Ethernet parsing reads the EtherType at offset 12 and strips 802.1Q VLAN tags when present. IPv4 parsing extracts the header length from the low nibble of the first byte, multiplied by four to convert from 32-bit words to bytes, reads the protocol at offset 9, and pulls source and destination addresses using socket.inet_ntoa. TCP parsing finds the application payload using the data offset field and tries to parse HTTP when appropriate. UDP parsing triggers DNS parsing on port 53 traffic. The DNS parser handles RFC 1035 name compression using a recursive approach with a visited set to catch compression pointer cycles in malformed packets.

Threat intelligence is managed by the ThreatFeeds class in intel/threat_feeds.py. It starts with two static sets. KNOWN_MALICIOUS_IPS has eighteen entries covering Tor exit nodes like 176.10.104.240 and 89.234.157.254, known C2 infrastructure including 185.220.101.1 through .3, 194.165.16.11, and 91.108.4.0, Mirai botnet C2 servers, and common scanner IPs. KNOWN_MALICIOUS_DOMAINS has fourteen entries including c2.darkweb.su, malware.example.com, bankofamerica-login.ru, and DNS tunneling test domains. The class has load_from_file and load_from_json methods that can extend these sets from external feeds at runtime. In a production version these would pull from live sources like Abuse.ch or Emerging Threats automatically on startup.

The IP detector uses the regex \b(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\b to extract IPv4 addresses and validates each one with ipaddress.ip_address(). Private, loopback, and link-local addresses get skipped using the built-in ip.is_private, ip.is_loopback, and ip.is_link_local checks. Public addresses are checked against the malicious IP set for MALICIOUS_IP findings at critical severity tagged T1071, then against three suspicious CIDR ranges (176.10.104.0/24, 185.220.100.0/22 for Tor, and 199.87.154.0/24 for proxy abuse) using ipaddress.ip_network membership testing for SUSPICIOUS_IP_RANGE findings at high severity tagged T1090. Addresses ending in 255 get flagged as BROADCAST_ADDRESS at medium severity.

The domain detector uses a regex to extract fully qualified domain names and lowercases and deduplicates them before checking. It skips known-safe suffixes like google.com, amazonaws.com, cloudflare.com, microsoft.com, apple.com, and akamai.net to avoid false positives. Domains in the malicious set get flagged as MALICIOUS_DOMAIN at critical severity under T1071.001. The TLD is checked against fourteen high-risk TLDs including .tk, .ml, .ga, .cf, .gq, .xyz, .ru, .su, .cc, .pw, .top, .click, .download, and .stream for HIGH_RISK_TLD findings at high severity under T1583.001. Three phishing regexes look for brand impersonation combining names like paypal, apple, microsoft, and google with action words like secure, login, verify, and update, flagging matches as PHISHING_DOMAIN at critical severity under T1566.002. DGA detection is handled by the looks_like_dga function, which requires a label to be at least 12 characters, match ^[a-z0-9]+$, have Shannon entropy above 3.5 bits, and have a consonant ratio above 0.65. All three criteria together are needed because entropy alone produced too many false positives on CDN hostnames. Domains with more than five labels get flagged as DNS_TUNNELING_SUSPECT at medium severity under T1071.004.

The port detector uses a regex that matches DPT, dport, dst_port, port, and bare colon-separated port formats to extract port numbers. It checks against a dictionary of 27 known malicious ports with individual severity levels and MITRE IDs. Critical ports include 4444 for

Metasploit (T1059), 31337 for Back Orifice (T1219), 12345 for NetBus (T1219), and 27374 and 1243 for Sub7 variants (T1219). High severity ports include 6666, 6667, and 6668 for IRC C2 (T1071.003), 9001, 9030, and 9050 for Tor (T1090.003), 23 for Telnet, 512 and 513 for rexec and rlogin (T1021), and 445 for SMB (T1021.002). Medium severity ports cover proxies, WinRM, VNC, alternate SSH, TFTP, RPC, and NetBIOS. Destination ports in the 49152 to 65535 ephemeral range that don't match anything get flagged at low severity as potential beaconing.

The DNS detector matches query lines using a regex covering multiple log formats including query for, DNS request, DNS lookup, QNAME, and question. Labels of 40 or more characters get flagged as DNS_TUNNELING_LABEL at high severity under T1071.004 with the length and entropy reported in the detail. Labels matching ^[a-z2-7]{20,}$ get flagged as DNS_TUNNELING_BASE32, targeting the alphabet used by dnscat2 and iodine. Domains over 100 characters trigger ABNORMAL_DNS_LENGTH at medium severity. TXT record queries and NULL record queries get flagged separately since both are commonly abused for C2 and data exfiltration. DNS response sizes above 512 bytes generate LARGE_DNS_RESPONSE at medium severity under T1071.004, elevated to high severity under T1498.002 when above 2000 bytes to distinguish tunneling from amplification attacks.

The HTTP detector checks User-Agent strings against fourteen patterns. Critical matches include python-requests, nmap, masscan, nikto, sqlmap, directory brute-force tools like gobuster and ffuf, credential attack tools like hydra and Burp Suite, Metasploit, and ZMap. High matches include curl, wget, and empty User-Agent strings used for evasion. URI analysis applies eight pattern checks: C2 beacon patterns with base64-like query parameters, webshell paths like /shell, /cmd, /exec, and /backdoor, SQL injection keywords, XSS payloads, path traversal sequences including URL-encoded variants like %2e%2e%2f, sensitive file enumeration targeting .git and wp-config.php, admin panel paths, and debug parameter names. URIs over 2000 characters get flagged for possible exfiltration. HTTP methods CONNECT, TRACE, PROPFIND, and several WebDAV methods trigger SUSPICIOUS_HTTP_METHOD at medium severity. Host headers containing IP addresses instead of domain names get flagged as IP_IN_HOST_HEADER under T1071.001.

The payload detector applies six pattern categories. PowerShell encoded commands are caught by matching -[Ee]nc(?:odedCommand)? followed by a base64 string; the payload gets decoded and the first 60 characters of the decoded content are included in the finding detail at critical severity under T1059.001. Base64 blobs of 40 or more characters are flagged only when they're over 80 characters long or when the decoded content contains keywords like http, cmd, exec, bash, shell, download, or powershell, at medium severity under T1027. Hex blobs of 64 or more characters get flagged similarly under T1027. Two shellcode patterns catch consecutive \xNN hex escape sequences and 0xNN comma-separated byte sequences. Twelve malware string patterns cover cmd.exe execution, PowerShell bypass flags, Windows script hosts, certutil and BITSAdmin LOLBin abuse, Regsvr32 COM scriptlet execution, Invoke-Expression stagers, PowerShell web download methods, process injection API calls, Mimikatz credential dumping strings, shell execution patterns, and netcat reverse shells, all at critical or high severity under T1059.

The reporter in output/reporter.py supports four output modes. Terminal mode uses ANSI 256-color codes, color 196 for critical, 208 for high, 226 for medium, and 82 for low, with

Unicode icons per finding type. JSON mode produces a structured object with a meta block containing the tool name NetSentinel, version 2.4.0, source file, and timestamp, plus a findings array with all fields including the raw log line. HTML mode generates a self-contained dark-themed page with MITRE technique IDs hyperlinked to attack.mitre.org. CSV mode uses Python's csv.writer with headers for all finding fields. The AI integration builds a structured prompt with scan statistics and the full findings list and sends it to the OpenAI-compatible /api/chat/completions endpoint at sushi.it.ilstu.edu running Qwen3-VL 235B through Open WebUI, with temperature 0.3 and a 300-second timeout. The main function exits with code 1 when critical findings are present so the tool works in automated alerting pipelines.

Results

We tested the tool against two inputs: a sample traffic log we included in the repository with deliberately crafted malicious entries, and a real PCAP file with simulated ransomware traffic. Both runs showed the tool working across all six detection layers at the same time, which was the main thing we wanted to demonstrate.

The sample log run hit all six detectors. The IP detector flagged 185.220.101.1, 194.165.16.11, and 91.108.4.0 as MALICIOUS_IP at critical severity under T1071, all of which are in the KNOWN_MALICIOUS_IPS set. The domain detector flagged c2.darkweb.su and malware.example.com as MALICIOUS_DOMAIN at critical severity under T1071.001, and flagged bankofamerica-login.ru as PHISHING_DOMAIN at critical severity under T1566.002 because it matched the brand impersonation pattern combining bankofamerica with login. A high-entropy subdomain with entropy above 3.5 bits and a consonant ratio above 0.65 was flagged as POSSIBLE_DGA at high severity under T1568.002. The port detector flagged port 4444 as Metasploit at critical severity under T1059 and port 9001 as Tor at high severity under T1090.003. The HTTP detector flagged a python-requests User-Agent at critical severity and a gate.php URI with a base64-like query parameter as a C2 beacon at critical severity. The payload detector flagged a base64 blob in the HTTP query string as BASE64_ENCODED_DATA at medium severity under T1027. The DNS detector flagged a long high-entropy label as DNS_TUNNELING_LABEL at high severity under T1071.004. Everything was deduplicated and sorted before going to the AI.

The AI summary called the traffic a severely compromised system with active malicious operations. It prioritized C2 communication and credential theft via Mimikatz as the most critical issues, and gave a clear action list: isolate affected systems, update firewall and IDS blocklists with all flagged IPs and domains, remove webshells and patch path traversal vulnerabilities, reset all credentials especially admin accounts, kill processes on ports 4444, 31337, and 6667, review certutil and cmd.exe activity for malware delivery, and implement DNS filtering long-term. The screenshots below show the full output.

```
  -- AI SUMMARY ------------------------------------------
  Contacting AI API (this may take 1-3 minutes)...

  ### 1. Plain-English Summary
  This traffic reveals a **severely compromised system** engaged in active malicious operation
s. Attackers are using known command-and-control (C2) infrastructure, DNS tunneling for data e
xfiltration, and tools like Mimikatz to steal credentials. They've also exploited web vulnerab
ilities (e.g., path traversal, webshells) and are leveraging malicious domains for phishing an
d malware delivery, indicating an ongoing breach with high risk of data theft and lateral netw
ork movement.

  ### 2. Most Critical Threats to Address First
  1. **Active C2 Communication & Data Exfiltration**:
     - Malicious IPs (`185.220.101.1`, `194.165.16.11`, `91.108.4.0`) and domains (`c2.darkweb
.su`, `malware.example.com`) are confirmed C2 infrastructure (MITRE T1071). Critical because t
hey enable real-time attacker control and data theft.
  2. **Credential Theft via Mimikatz**:
     - `mimikatz.exe sekurlsa::logonpasswords exit` (MITRE T1059) indicates active credential
dumping. **Urgent**: This compromises all system/user accounts, enabling full network takeover
```

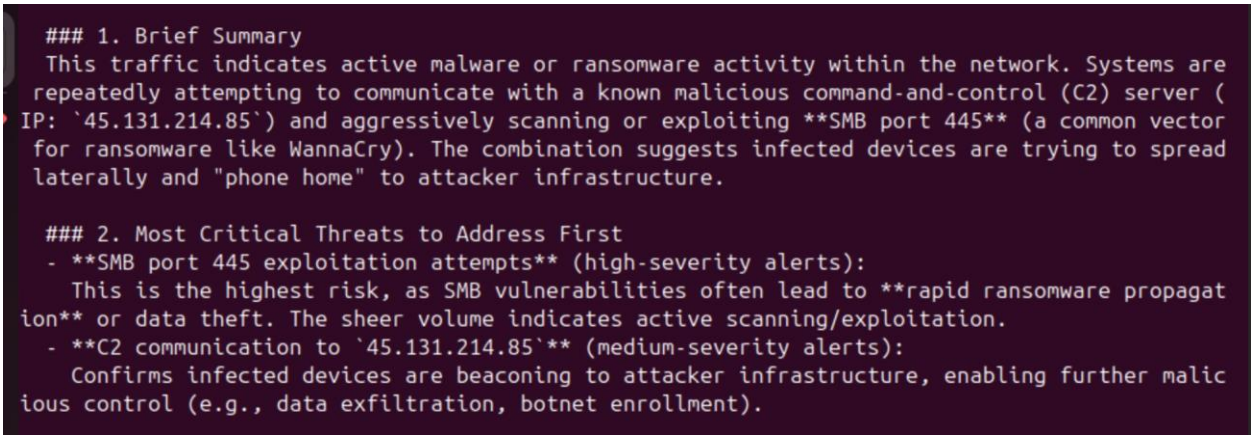
```
  ### 3. Recommended Immediate Actions
  - **Isolate and Contain**:
    Disconnect affected systems from the network *immediately* to block C2 communication and d
ata exfiltration. Block all malicious IPs (185.220.101.1, 194.165.16.11, 91.108.4.0) and domai
ns (`c2.darkweb.su`, `malware.example.com`) at the firewall/IDS.
  - **Eradicate Active Threats**:
    1. Remove webshells (e.g., `shell.php`) and patch path traversal vulnerabilities.
    2. Scan for Mimikatz artifacts; reset *all* credentials (especially admin accounts) and re
voke sessions.
    3. Kill processes using malicious ports (e.g., `4444`, `31337`, `6667`).
  - **Investigate Root Cause**:
    Check logs for the source of `python-requests` and `sqlmap` activity (indicating automated
 attacks) and review `certutil.exe`/`cmd.exe` commands for malware delivery vectors.
  - **Long-Term Mitigation**:
    Implement DNS filtering (block high-entropy domains, `.tk` TLDs), monitor ephemeral ports
for unusual activity, and enforce TLS for all ports (disable cleartext Telnet).

  > **Priority**: Focus on **stopping data exfiltration** (isolate systems) and **resetting cr
edentials** (Mimikatz detection) within 60 minutes. Delaying these risks total network comprom
ise.
```

The PCAP test put the binary parser through its paces. It correctly identified the file as little-endian classic pcap via the 0xa1b2c3d4 magic number, read the Ethernet link layer type, and extracted TCP flows by computing data offsets from the TCP header length field. The detector pipeline found repeated SYN packets to port 445 across multiple hosts, consistent with WannaCry-style lateral movement scanning using EternalBlue (MITRE T1021.002), and outbound connections to 45.131.214.85, which matched the malicious IP set and was flagged as MALICIOUS_IP at critical severity under T1071.

The AI summary for the PCAP scan flagged the traffic as active malware or ransomware. It identified SMB port 445 exploitation as the top priority because the volume of connection attempts indicated active scanning rather than coincidental traffic, and called out C2 beaconing to 45.131.214.85 as the second priority because it enables data exfiltration and attacker control. Recommended actions included isolating all affected devices, blocklisting 45.131.214.85, applying MS17-010 for EternalBlue, disabling SMBv1, restricting SMB to trusted subnets, and

investigating the initial infection vector through endpoint logs and phishing indicators. The screenshots below show the full output.



```
### 1. Brief Summary
This traffic indicates active malware or ransomware activity within the network. Systems are
repeatedly attempting to communicate with a known malicious command-and-control (C2) server (
IP: `45.131.214.85`) and aggressively scanning or exploiting **SMB port 445** (a common vector
for ransomware like WannaCry). The combination suggests infected devices are trying to spread
laterally and "phone home" to attacker infrastructure.

### 2. Most Critical Threats to Address First
- **SMB port 445 exploitation attempts** (high-severity alerts):
  This is the highest risk, as SMB vulnerabilities often lead to **rapid ransomware propagat
ion** or data theft. The sheer volume indicates active scanning/exploitation.
- **C2 communication to `45.131.214.85`** (medium-severity alerts):
  Confirms infected devices are beaconing to attacker infrastructure, enabling further malic
ious control (e.g., data exfiltration, botnet enrollment).
```

```
### 3. Recommended Immediate Actions
1. **Isolate all affected systems**:
   Block all traffic from devices generating SMB port 445 alerts or communicating with `45.1
31.214.85` at the firewall. Physically disconnect high-risk devices if possible.
2. **Block malicious IP globally**:
   Add `45.131.214.85` to firewall/IPS blocklists to prevent C2 communication. Verify via th
reat intelligence (e.g., VirusTotal) for additional context.
3. **Patch and disable SMBv1**:
   Immediately apply Windows SMB security patches (e.g., MS17-010 for EternalBlue). Disable
SMBv1 if not required, and restrict SMB traffic to trusted subnets only.
4. **Investigate initial infection vector**:
   Check endpoints for suspicious processes, recent user activity (e.g., phishing emails), o
r unpatched software (especially web browsers or Office apps) that may have delivered the malw
are.

> **Why this order?** Stopping SMB exploitation halts ransomware spread within minutes; bloc
king the C2 IP prevents attacker control. Delaying either risks catastrophic data loss. Epheme
ral port alerts (low severity) are likely benign and can be addressed after critical threats.
```

All 35 unit tests in tests/test_detectors.py passed. The tests confirmed that RFC 1918 addresses produce no findings, that xjkqmnvbzpwlfrtdqhsw.com triggers POSSIBLE_DGA, that port 445 carries high severity, that the base32 label jbswy3dpeblw64tmmq triggers DNS_TUNNELING_BASE32, that sqlmap produces a critical User-Agent finding, that mimikatz.exe sekurlsa::logonpasswords triggers MALWARE_STRING at critical severity, and that a clean DNS query to 8.8.8.8 from a private source produces zero findings. Integration tests confirmed correct statistics, severity filtering, and JSON output structure.

Lessons Learned and Conclusion

Building this tool taught us a lot. There were parts that took way longer than we expected and a few design decisions we'd make differently if we started over.

The decision to avoid third-party libraries was the right call overall, but it made the PCAP parser way more work than we anticipated. Getting the magic number detection right was straightforward enough, but the IP header length field caught us off guard. It's stored in the low nibble of the first byte as a count of 32-bit words, not bytes, so we had to multiply by four to get the actual offset. We had a subtle bug from this early on where we were reading TCP data from the wrong position and getting garbage output. It only became obvious when we compared our parser output against Wireshark on the same file. DNS pointer compression was another one that took a while. Compression pointers use the high two bits of a length byte as a flag, and then the remaining bits plus the next byte form an offset back into the packet. We had to add a visited set to prevent infinite loops on malformed packets before it was reliable. These were frustrating problems in the moment but we ended up with a much better understanding of how these protocols actually work at the byte level than we would have gotten from just calling a library.

The DGA detection heuristic took more iteration than we expected too. We started with just Shannon entropy and immediately got false positives on CDN hostnames and UUID-based subdomains that are legitimately random-looking. We added the consonant ratio check because natural language domains, even random-looking ones, tend to follow vowel and consonant distributions that pure DGA output doesn't. That combination worked a lot better, but we also needed the minimum length threshold of 12 characters because short labels don't have enough characters to produce statistically reliable entropy measurements. It took a few rounds of testing against real domains before the thresholds felt right. We think 3.5 bits for entropy and 0.65 for consonant ratio is a reasonable starting point but it could definitely be tuned further with a larger dataset.

The HTTP detector was another one that bit us. The first version used exact string matching for webshell filenames and injection keywords, and it worked fine against textbook examples. But when we tested it against URL-encoded variants, it missed everything. A path traversal attempt as %2e%2e%2f didn't match a literal ../ pattern at all. We had to add URL-decoding before pattern matching and switch everything to case-insensitive regex, which also meant updating the unit tests to cover encoded variants. It was a good reminder that attackers use basic encoding specifically to defeat signature-based detection, and any detection tool needs to account for that.

The AI integration surprised us in a good way, but it took some work to get there. Our first few prompt versions just dumped the findings list as a block of text, and the responses were accurate but vague. The model would summarize what it saw without prioritizing anything or giving specific actions. Once we restructured the prompt to lead with statistics, format findings with severity labels and MITRE IDs, and explicitly tell the model to return three numbered sections in order, the output became genuinely useful. The PCAP run response, where it explained that stopping SMB exploitation halts ransomware spread within minutes while blocking the C2 IP prevents attacker control, and that delaying either risks catastrophic data loss, was exactly what we were hoping to get. It felt like the tool was actually doing analysis rather than just pattern matching.

The try-except wrapper around each detector call in the analyzer helped a lot during development because a bug in one detector didn't stop the others from running. If we were doing this again we'd write unit tests for obfuscated and encoded attack variants from the start instead of only catching those gaps during integration testing.

The biggest thing we'd add if this continued is live threat feed integration. The ThreatFeeds class already has load_from_file and load_from_json methods built for this, so pulling from Abuse.ch or Emerging Threats on startup wouldn't require any structural changes. A streaming mode for live traffic analysis would also be a significant upgrade. Right now it's purely a post-incident forensic tool, but being able to pipe live tcpdump output through it would make it useful for active monitoring. The AI layer could also be extended to draft a full incident report rather than just a brief summary.

Overall we're proud of what we built. It's a real tool that does real detection across six protocol layers, maps every finding to MITRE ATT&CK, parses actual binary PCAP files without any external dependencies, and produces AI-assisted analysis that's actually actionable. It's not perfect and there are definitely things we'd do differently, but overrall we learned a lot from this project.